bcc09

A C compiler and tools for the Motorola 6809 – user's manual

bcc09 project

November 2026

Contents

1 Introduction **2**

2 Building and installing **3**

3 Getting started **3**

3.1 For Color BASIC . . . 3

3.2 For OS-9 . . . 4

3.3 For FLEX . . . 4

3.4 For a monitor, or a machine of one's own . . . 4

4 The C language **4**

5 Targets **5**

5.1 Lines . . . 5

5.2 Standalone (`-Ms`) . . . 6

5.3 Color BASIC (`-Me`) . . . 6

5.4 ROM cartridges (`-Mr`) . . . 6

5.5 Vectrex (`-Mv`) . . . 7

5.6 OS-9 (`-Mo`) . . . 8

5.7 FLEX (`-Mf`) . . . 8

5.8 sim6809 (`-Mi`) . . . 8

6 The C library **9**

7 Calling conventions and assembly **9**

7.1 Functions . . . 9

7.2 An assembly function . . . 9

7.3 Run-time helpers . . . 10

7.4 Init functions and segments . . . 10

8 The tools **10**

8.1 Linker scripts 11

9 Testing **11**

Appendix A: the manual pages **12**

bcc09(1) 12

as09(1) 16

ld09(1) 18

ar09(1) 23

objdump09(1) 24

objcopy09(1) 25

bcc09-cpp(1) 26

bcc09-unproto(1) 27

bcc09-cc1(1) 30

bcc09-copt(1) 31

1 Introduction

bcc09 compiles C for the Motorola MC6809, the 8-bit processor of the Tandy Color Computer, the Dragon 32 and 64, and many boards and systems of the 1980s and since. It comes with an assembler, a linker, a librarian, tools to look into and convert the files they make, and a C library for the systems a program can run on:

- **Color BASIC** (and Extended and Disk BASIC) on the CoCo 1, 2 and 3 and the Dragon: programs loaded with `LOADM` or `CLOADM` and run with `EXEC`, with the keyboard, the screen, and (on a CoCo) Disk BASIC's files.
- **OS-9** Level 1 and 2, and NitrOS-9: position-independent, reentrant program modules, with files, directories and a heap that grows.
- **FLEX**: commands (`.CMD` files), with the terminal and FLEX's files.
- **Standalone** code, loaded and called by a monitor or a loader.
- **sim6809**, the simulator bcc09's own tests run on.

bcc09 began as the 6809 configuration of Bruce Evans's C compiler from the dev86 package, the tools Linux-8086 and ELKS were built with. dev86's 6809 compiler was never finished; bcc09 finishes it, fixes it, and adds the C library, the targets, and a test suite that runs the programs it compiles, on a simulator and on real ROMs and systems in the XRoar emulator.

This manual describes how to use it: building and installing it (chapter 2), compiling a first program for each target (3), the C it compiles (4), each target (5), the C library (6), how compiled code calls functions, so that assembly can be written to work with it (7), and the tools (8). For a gentler start, "bcc09 for BASIC Programmers" (`basic.pdf`) teaches C from Color BASIC. Appendix A is the reference: the manual pages of every program.

bcc09 is free software. The compiler and tools are under the GNU General Public License; the C library, which is linked into the programs bcc09 builds, is under the GNU Library General Public License, version 2, so that those programs can be under any license.

2 Building and installing

bcc09 builds on any Unix-like system with a C compiler and `make`:

```
make
make check
make install PREFIX=/usr/local/bcc09
```

`make` builds the programs and the C library (which bcc09 compiles itself). `make check` runs the regression tests (chapter 9). `make install` puts:

- the programs `bcc09`, `as09`, `ld09`, `ar09`, `objdump09` (also installed as `nm09` and `size09`) and `objcopy09` in *PREFIX*`/bin`;
- the compiler's passes (`cpp`, `unproto`, `cc1`, `copt` and its rules) in *PREFIX*`/libexec/bcc09`;
- the C library, the targets' start-up code and libraries in *PREFIX*`/share/bcc09`, and its headers in *PREFIX*`/share/bcc09/include`;
- the manual pages in *PREFIX*`/share/man/man1`.

PREFIX is `/usr/local/bcc09` by default. bcc09 finds its passes and library under the *PREFIX* it was built with, or under the directory the environment variable `BCC_PREFIX` names: so a build tree can be used without installing it, by pointing `BCC_PREFIX` at a directory of links to it, as the tests do.

`make install` also takes `DESTDIR`, a directory to install into as if it were the root, for making packages: bcc09 still looks for its files under *PREFIX*. The host compiler's options can be set with `CFLAGS` and `LDFLAGS` (in the environment or on `make`'s command line); `-std=gnu89` and what each program needs are always added.

On Fedora, `make rpm` builds the RPM packages from `bcc09.spec`, with *PREFIX* `/usr`, in `rpmbuild/`: `bcc09`, and `bcc09-doc` (noarch) with this manual and "bcc09 for BASIC Programmers" as PDFs, from the release's tag (`v0.12.0`), or from the last commit if that isn't tagged yet. `make srpm` builds the source package alone.

3 Getting started

A first program:

```
#include <stdio.h>

int main()
{
    printf("HELLO, WORLD\r");
    return 0;
}
```

The `\r` is not a mistake: Color BASIC, OS-9 and FLEX end a line with a carriage return, and bcc09's C library passes characters to them as they are (section 5.1).

3.1 For Color BASIC

```
bcc09 -Me -O -o hello -X-T0x3000 hello.c
objcopy09 -O cas -n HELLO hello hello.cas
```

`-Me` chooses the Color BASIC target, `-O` the optimizer, and `-X-T0x3000` gives the linker the load address, low in memory, below BASIC's stack and strings. `hello` is an executable (an a.out file); `objcopy09` makes it a cassette image. In an emulator, or played to a real CoCo or Dragon:

```
CLOADM:EXEC
```

For a disk, `objcopy09 -O decb hello HELLO.BIN` makes a file for `LOADM`.

3.2 For OS-9

```
bcc09 -Mo -O -o hello.out hello.c
objcopy09 -O os9 -n hello hello.out hello
```

`hello` is a program module: copy it to a disk's `CMDS` directory (with the execute attributes set), and run it from the shell.

3.3 For FLEX

FLEX's terminal wants a line feed after the carriage return, so the program prints `"HELLO, WORLD\r\n"`. Then:

```
bcc09 -Mf -O -o hello -X-T0x1000 hello.c
objcopy09 -O flex hello HELLO.CMD
```

Put `HELLO.CMD` on a FLEX disk, and type `HELLO`.

3.4 For a monitor, or a machine of one's own

```
bcc09 -O -o prog -X-T0x4000 prog.c
objcopy09 -O srec prog prog.s19
```

The default target, `-Ms`, has no I/O: `main` is called, and its result is returned in D to whatever called the program. Section 5.2 tells how to give such a program I/O.

4 The C language

bcc09 compiles C89, with these sizes:

Type	Size	Notes
`char`	8 bits	unsigned
`short`, `int`	16 bits	
`long`	32 bits	
pointers	16 bits	
`float`	32 bits	IEEE 754 single precision, big-endian
`double`	64 bits	IEEE 754 double precision, big-endian
`_Bool`	8 bits	C99's: any value converted to it is 0 or 1

Numbers are big-endian, like the 6809's: the most significant byte first. There is no 64-bit integer type.

- **Prototypes** work as C89 says: a call converts each argument to its parameter's type, through a pointer to a function too, and a call with the wrong number of arguments is an error. (A pass called `unproto` rewrites ANSI C to the older form the compiler reads, and tells it the parameters' types; `-kr` leaves it out, for K&R C.)
- **`const` and `volatile`** aren't known to the compiler: the C library's headers define them away, so a program that uses them must include one.
- **Floating point** is in software, exact: every operation gives the correctly rounded result IEEE 754 calls for, the same as a host's hardware, with infinities, NaNs and denormals. `float` arithmetic is done in `double`, and rounded when it is stored. On a CoCo (0.89 MHz) a `double` addition takes about 2 ms, a multiplication 5 ms and a division 9 ms; `float`'s about 3, 4 and 10 ms.
- **Constants:** with a 16-bit `int`, a decimal constant over 32767 is a `long`, as C says; write `40000U` for an `unsigned int`.
- **What C leaves undefined** is as follows: signed division truncates toward zero; dividing by zero doesn't trap; shifting by the operand's width or more gives 0 (or copies of the sign bit for a signed right shift); a floating value converted to an integer type it doesn't fit gives its low bits.

The preprocessor defines `__BCC__`, `__6809__`, `__STDC__` (unless `-kr`), and a macro for the target: `__CB__`, `__OS9__`, `__FLEX__` and `__FLEX9__`, `__STANDALONE_`, or `__SIM6809__`.

5 Targets

bcc09's `-M` option chooses what a program is for. Each target has its own start-up code and a library of the functions that differ (input, output, files, the end of the program), which `bcc09` links with the C library:

Option	Target	Start-up, library	Output
`-Ms`	standalone (default)	`crt0.o`, `libsa.a`	any (`objcopy09`)
`-Me`	Color BASIC, run with `EXEC`	`crt0.o`, `libcb.a`	`cas`, `wav`, `decb`
`-Mr`	ROM cartridge (CoCo, Dragon)	`crtrom.o`, `librom.a`, `rom.ld`	the ROM's image
`-Mv`	Vectrex cartridge	`crtvec.o`, `libvec.a`, `vectrex.ld`	the ROM's image
`-Mo`	OS-9 program module	`crtos9.o`, `libos9.a`	`os9`
`-Mf`	FLEX command	`crtflex.o`, `libflex.a`	`flex`
`-Mi`	sim6809	`crtsim.o`, `libsim.a`	`srec`

(`-Mu` and `-Mb`, for Color BASIC `USR` functions and Disk BASIC, define their macros but have no start-up code yet.)

Every target's start-up code clears the BSS, calls the *init functions* (section 7.4), then `main`; `main`'s result goes to `exit()`, which calls the functions `atexit()` registered, last first, and then the target's `_exit()`. The heap, which `malloc` uses, lies between the end of the program and its stack.

5.1 Lines

Color BASIC, OS-9 and FLEX end a line of text with a carriage return (CR, `'\r'`), not C's `'\n'`; FLEX's terminal also wants a line feed after it to start a new line. bcc09's C library doesn't translate: `read()` and `write()` pass characters as they are. So a program for those systems prints `"\r"` (on FLEX, `"\r\n"`) to end a line, and reads lines ending in `'\r'`. `fgets()` stops only at `'\n'`; POSIX's `getdelim()` reads up to any character:

```
char *line = 0;
size_t size = 0;

while (getdelim(&line, &size, '\r', stdin) > 0)
    /* line is the line, with its '\r' */ ;
```

5.2 Standalone (-Ms)

The program runs on the stack its loader gave it, and returns to it: `main`'s result (or `exit()`'s argument) is in D. It has no I/O: `read()` and `write()` aren't there. To give it I/O, write them (section 7), and link with `-x`, which leaves out the start-up code, and a start-up file of one's own.

5.3 Color BASIC (-Me)

The program runs on BASIC's stack and returns to BASIC. Load it low (`-X-T0x3000`, say), below BASIC's stack and string space at the top of memory; the heap is the memory between the end of the program and the stack.

- **Output:** `write()` (and so `printf()`) prints on the screen through the ROM's `CHROUT`.
- **Input:** `read()` reads a line from the keyboard through `POLCAT` as a terminal does: the keys are shown as they are typed, the left arrow rubs out, and ENTER ends the line (with a CR). BREAK at the start of a line is the end of the input.
- **Files,** with Disk BASIC on a CoCo: `open()` and `unlink()`, and `read()`, `write()`, `lseek()` and `close()` on what `open()` gives, so `fopen()` too. Names are `NAME.EXT` (or `NAME/EXT`), with `:D` for drive *D*. bcc09 reads and writes the disk's sectors through Disk BASIC's `DSKCON`, keeping the FAT and directory as Disk BASIC does: a new file is a data file, ASCII, its granules chosen as Disk BASIC chooses them, so BASIC and a program can share a disk. On a full disk, `write()` writes what fits and then fails with `ENOSPC`. `lseek()` can't go past a file's end. A program that opens no files doesn't carry this code.

The Dragon's ROM has the same `POLCAT` and `CHROUT`, so programs run on the Dragon 32 and 64 too, but without files: DragonDOS's disks are of another format, and `open()` fails with `ENODEV` there.

`coco.h` has the keyboard, the text screen and the joysticks as BASIC has them, for `-Me` and `-Mr` programs: `inkey()` (`INKEY$`: the key pressed, or 0), `cls(c)` (`CLS c`; -1 for plain `CLS`), `locate(row, col)` (where `PRINT@` would print: `PRINT@ n` is `locate(n / 32, n % 32)`), `joystk(n)` (`JOYSTK`), `sound(tone, duration)` (`SOUND`: BASIC's own tone loop, so its pitches, with the duration in fifteenths of a second counted in periods of the tone, so that it works with the interrupts off), and `peek()` and `poke()`. stdout is written out at the end of each line (a `'\r'` as well as a `'\n'`), before input is read, and before `cls()` and `locate()` move the cursor.

5.4 ROM cartridges (-Mr)

A program in a cartridge's ROM, which the CoCo and the Dragon start by themselves (with the cartridge's CART line wired to Q, as in a game cartridge): BASIC's FIRQ handler jumps to `$C000`. `bcc09 -Mr` links it with the linker script `rom.ld` (section 8.1), and its output is the ROM's image, ready for an EPROM or for XRoar:

```
bcc09 -Mr -O -o hello.rom hello.c
xroar -cart-rom hello.rom -cart-autorun
```

- **The ROM,** from `$C000`: the code, then an image of the initialized data, filled out with `$FF` to 8K, or to 16K less the last page (`$FF00` up is I/O).

- **The RAM:** the data, the BSS and the heap from `$0600`, after the text screen; the stack below `$8000`, a 32K machine's top (`-X-A__stack_top=0x4000` for 16K).
- **Start-up:** with the interrupts off (and the cartridge's FIRQ turned off), the start-up code sets the stack, copies the initialized data to RAM, clears the BSS, and calls the init functions and `main`.
- **I/O,** as with `-Me` (but no files): the screen and keyboard through the ROM's `CHROUT` and `POLCAT`.
- **The end:** a cartridge has nothing to return to, so `_exit()` stops, with the status in D.

The CoCo 3 starts a cartridge before its BASIC is ready for `CHROUT` and `POLCAT`, so `-Mr`'s I/O doesn't work there. For another layout, copy `rom.ld` (from bcc09's library directory), change it, and link with `-X-S` and the copy.

5.5 Vectrex (`-Mv`)

A Vectrex cartridge. The BIOS checks the cartridge's header, shows its title, plays its tune, and then runs the code after the header:

```
#include <vectrex.h>

int main()
{
    for (;;) {
        Wait_Recal();
        Intensity_a(0x7F);
        Print_Str_d(0, -40, "HELLO\200");
        Reset0Ref();
        Moveto_d(-30, -30);
        Draw_Line_d(0, 60);
    }
}
```

```
bcc09 -Mv -O -o hello.bin hello.c
mame vectrex -cart hello.bin
```

- **The ROM,** from `$0000`: the header, the code, and an image of the initialized data (up to 32K).
- **The RAM** is the Vectrex's 1K, less the BIOS's own variables: the data and BSS from `$C880`, the stack below `$CBEA`. That is room for a game's variables, but not for stdio's buffers, and the heap is small.
- **Start-up:** the start-up code sets the stack and DP (`$D0`, where the BIOS's routines want it), copies the initialized data to RAM, clears the BSS, and calls the init functions and `main`.
- **The BIOS:** `vectrex.h` declares its routines for drawing (`Wait_Recal`, `Intensity_a`, `Moveto_d`, `Draw_Line_d`, `Dot_d`, `Draw_VLc`, `Print_Str_d`, `Reset0Ref`), the controllers (`Read_Btns`, `Joy_Digital`), sound (`Init_Music_chk`, `Do_Sound`) and `Random`, with the names Vectrex programmers know, and its variables (`Vec_Btn_State`, `Vec_Joy_1_X`, `VIA_t1_cnt_lo` for the scale, and others). Coordinates are -128 to 127, y first; the BIOS's strings are in capitals and end with `\200` (`$80`), not 0.
- **No other I/O:** `read()` and `write()` aren't there, so neither is stdio.
- **The end:** `_exit()` stops, with the status in D.

The header is `libvec.a`'s, with the title "BCC09", unless the program has its own: `vectrex.h` shows one, an assembly file of a few lines, linked with the program (`bcc09 -Mv -o game.bin game.c title.s`). For another layout, copy `vectrex.ld`, change it, and link with `-X-S` and the copy.

5.6 OS-9 (-Mo)

The program is an OS-9 program module: its code is position-independent (LBSR for calls, ,PCR for constants), and its data is reached through the U register, which points at the data area F$Fork gives each process, so the module is reentrant: one copy in memory serves every process running it. bcc09 -Mo builds the whole C library this way.

```
bcc09 -Mo -O -o prog.out prog.c
objcopy09 -O os9 -n prog -d 2048 prog.out prog
```

-d sets the stack's size (1K by default). The start-up code copies the data's initial values from the module to the data area, clears the BSS, and runs the code that sets static variables holding addresses (which aren't known until the program runs). main gets the command line's words (argv[0] is the module's name; "..." keeps words together).

- **I/O:** read() and write() use I$ReadLn and I$WritLn, which a terminal edits and echoes a line at a time; read() gives at most a line, up to its CR.
- **Files:** open() (with O_CREAT, O_EXCL, O_TRUNC and O_APPEND), lseek(), unlink(), remove().
- **Directories:** mkdir(), chdir(), rmdir(), getcwd(), and opendir(), readdir(), rewinddir() and closedir() (dirent.h). OS-9 has no calls for rmdir() and getcwd(), which do what its deldir and pwd do: rmdir() checks that the directory is empty, turns off its D attribute (SS.Attr, NitrOS-9's and Level 2's) and deletes it; getcwd() goes up through .. to the device's root, and gives a path such as /D1/USR/SRC.
- **The heap** grows with F$Mem beyond the data area F$Fork gave (Level 1 can only grow it if the memory after it is free).
- **errno:** OS-9's errors become ENOENT, EBADF, EACCES, EEXIST, ENOMEM, or EIO.

5.7 FLEX (-Mf)

The program is a FLEX command. Its start-up code moves the stack from FLEX's own small one to the end of user memory, and gives main the command line's words: argv[0] is the command's name as typed ("P,PROG", which prints PROG's output, gives PROG), and the rest are the words after it, separated by blanks or commas.

- **Output** goes through FLEX's PUTCHR, so FLEX's P and O commands redirect it; **input** comes a line at a time from INCH, echoed, with TTYSET's backspace character to rub out.
- **Files**, through FLEX's File Management System: open(), unlink() and rename(). FLEX writes only new files from their start: writing to an existing file needs O_TRUNC (which fopen(name, "w") gives), which replaces it; O_RDWR and O_APPEND aren't possible. Files are text, with FLEX's space compression, unless opened in binary (O_BINARY, or "b" in fopen()'s mode); a binary file reads back with the zeros that pad its last sector, since FLEX keeps a file's size only in sectors. lseek() works on a file open for reading.
- **The end:** _exit() closes every file and returns to FLEX (FLEX has no exit status).

5.8 sim6809 (-Mi)

For sim6809, the simulator bcc09's tests use (from https://github.com/retrotinker/sim6809): standard output goes to the simulator's output, standard input comes from its input, and main's result becomes the simulator's exit status. There are no files.

6 The C library

The headers are `ctype.h`, `dirent.h`, `errno.h`, `fcntl.h`, `limits.h`, `malloc.h`, `math.h`, `stdarg.h`, `stdbool.h`, `stddef.h`, `stdint.h`, `stdio.h`, `stdlib.h`, `string.h`, `unistd.h`, `sys/stat.h` and `sys/types.h`. Most of the library is dev86's, fixed; the parts that needed it were written for bcc09. Each function is a module of its own, so a program carries only what it uses.

- **stdio:** the `FILE` functions; the `printf` and `scanf` families, which behave as the GNU C library's do; and POSIX's `getline()` and `getdelim()`.
- **Floating point in text:** `printf`'s `%e`, `%f` and `%g` print the exact decimal value of a double, `%a` its bits in hexadecimal (`0x1.8p+1`), and `strtod()`, `atof()` and `scanf` give the double nearest the number they read, as the GNU C library's do. This code is linked only into programs that print or read a floating point number.
- **math.h:** the C89 functions, from Sun's fdlibm, whose results are bit for bit what the same code gives on any IEEE host; `errno` is set as the GNU C library sets it.
- **stdint.h** has the 8-, 16- and 32-bit types; **stdbool.h**'s `bool` is the compiler's `_Bool`.
- **malloc** gets memory from `sbrk()`. Except on OS-9, the heap may not come within `__heap_margin` bytes (256) of the stack pointer: a program that needs a deeper stack after filling its heap should raise it.

What is still to come: `div()` and `ldiv()`, and the functions of later C standards beyond those named here.

7 Calling conventions and assembly

7.1 Functions

A C function's first argument is passed in X (a `long`'s in D and X, high word in D); the others are on the stack, above the return address, in order, and the caller removes them. A `float` argument is passed as a `double`, and a `double` is always on the stack, even as the first argument. The result:

Type	Result in
`char`, `int`, pointer	X
`long`, `float`	D and X (high word in D)
`double`	D, X, Y and U (high word in D; in memory, as `PSHS D,X,U,Y` leaves them)

The caller saves the registers it needs: a function may change them all (but U under `-Mo`, where it points at the data). A function name in C is the assembler's name with an underscore before it: `main` is `_main`.

A function with a variable argument list gets, in place of `...`, an `int` parameter named `__va_alist`, which `stdarg.h`'s `va_start` uses: the first argument comes in a register, so the address of the last named parameter doesn't lead to the rest.

7.2 An assembly function

```
; int twice(int n): n * 2
        .text
        export  _twice
_twice: tfr     x,d             ; n, the first argument
        lslb
```

```
        rola
        tfr     d,x             ; the result, in X
        rts
```

Assembled with `as09 -o twice.o twice.s`, it links with C code that declares `int twice(int n);`.

7.3 Run-time helpers

The compiler calls helpers in the C library for what the 6809 can't do in a few instructions: 16-bit multiplication, division and shifts (`imul`, `idiv`, `isl` and the rest: operands in D and X, result in D), 32-bit arithmetic (`ladd_b`, `lmul_b` and the rest: the left operand in D:X, Y pointing at the right, the result in D:X), and floating point (`Fadd`, `Fmul`, conversions and the rest). The int and floating point helpers keep Y and U, the long ones U. The C library's README lists them all.

7.4 Init functions and segments

The assembler's 16 segments (`LOC` 0 to 15) are placed by the linker: 0 and 1 are code, 3 the data and then the BSS (`.data` and `.bss`). A module can have a function run before `main` by putting its address in segment 5:

```
loc 5
fdb myinit
```

The start-up code calls each one (skipping the zeros the linker pads the segment with). That is how the floating point formatting is linked into programs that need it, and how OS-9 programs set static variables that hold addresses.

8 The tools

- **`as09`**, the assembler: Motorola syntax (`LDA #$10`, `LEAX 4,S`, `FCB`, `FDB`, `FCC`, `RMB`, `EQU`), `export` and `import` for symbols shared between modules, `.text`, `.data` and `.bss`, conditionals and macros. Numbers are `$` or `0x` hex, `%` binary, decimal, or octal with a leading 0.
- **`ld09`**, the linker: links object files and libraries into an executable (an a.out file), loading only the modules something refers to. `-T` gives the code's address (hexadecimal), `-i` puts the data at 0 (OS-9's way), `-m` and `-M` list the modules and the layout, and `-S` takes a linker script (section 8.1).
- **`ar09`** makes libraries: `ar09 rc libmine.a one.o two.o`.
- **`objdump09`**, `nm09` and `size09` show what is in object files, libraries and executables.
- **`objcopy09`** writes an executable as what a machine loads:

Format	For
`binary`	the bytes alone
`srec`, `ihex`	Motorola S-records, Intel hex: EPROM programmers, monitors
`decb`	Disk Extended Color BASIC's `LOADM`
`cas`, `wav`	Color BASIC's `CLOADM`, as a cassette image or its sound
`ddos`	DragonDOS
`os9`	an OS-9 program module

Format	For
`flex`	a FLEX command (`.CMD`)

Appendix A has each program's options in full.

8.1 Linker scripts

A linker script says where the linker puts each part of a program. Without one, `ld09` uses its default script, which `ld09 -P` prints (with the `-T`, `-D` and `-H` given): the code at the text address, the data straight after it, and the BSS after the data. A script of one's own can put the code and data anywhere, in any order, store data in ROM to be copied to RAM, fill gaps, define symbols, and write S-records or Intel hex itself. `ld09(1)` describes the language; it comes from `ld01`, the linker of the mc6801-tools project, where it is GNU ld's in a smaller form.

This one puts a Color BASIC program's code low in memory, at `$0E00`, and its data and BSS at `$6000`, out of the way:

```
SECTIONS
{
    .text 0x0E00 : { *(LOC0 LOC1) }
    .data 0x6000 :
    {
        *(LOC2 LOC3 LOC4)
        *(LOC5)
        *(LOC6 LOC7 LOC8 LOC9 LOC1?)
        *(COMMON)
    }
}
```

```
bcc09 -Me -O -o prog -X-Ssplit.ld prog.c
objcopy09 -O decb prog PROG.BIN
```

`LOCn` is segment n of every module, and `COMMON` the common blocks (uninitialized variables). The script needn't define anything for the C library: `ld09` provides the symbols the start-up code uses (`__edata` and `__end`, between which it clears the BSS, and `__seg5DL` and `__seg5DH` around the init functions). The init functions must be in one place, so segment 5 has a statement of its own: a statement takes its input module by module. `objcopy09 -O decb` writes the code and the data as two blocks, which `LOADM` puts in their places.

9 Testing

`make check` runs bcc09's regression tests: it compiles the test sources with this build's tools, runs the programs on sim6809 (1.2.0 or later, from https://github.com/retrotinker/sim6809, on `PATH` or named by `SIM6809`; for every target: Color BASIC with stand-ins for its ROMs, OS-9 modules and FLEX commands with the simulator's emulation of their systems, cartridges with stand-ins for BASIC's ROM and the Vectrex's BIOS), compares every result with a reference copy, and checks the manual pages. Many of the programs are also compiled with the host's C compiler, and must print the same.

Four more test suites run programs on real ROMs and systems, in the XRoar and MAME emulators, if they and the ROMs and disks they need are there:

- `make check-xroar`: Color BASIC on the CoCo 1, 2 and 3 and the Dragon 32 and 64, with Disk BASIC's files on the CoCos, and ROM cartridges;
- `make check-nitros9`: OS-9 modules on NitrOS-9 Level 2;
- `make check-flex`: FLEX commands, and FLEX's files, on Compusense FLEX for the Dragon 64;
- `make check-vectrex`: a Vectrex cartridge on the Vectrex's BIOS in MAME (`$VECTREX_ROM`, or MAME's own ROM set).

Appendix A: the manual pages

The pages that `man` shows for each program follow.

bcc09(1)

NAME

bcc09 - C compiler for the Motorola 6809

SYNOPSIS

bcc09 [**-c**|**-S**|**-E**|**-P**] [**-O**] [**-M** *target*] [**-o** *output*] [*options*] *file...*

DESCRIPTION

bcc09 compiles C for the Motorola MC6809: the Tandy Color Computer and the Dragon under Color BASIC, OS-9, FLEX, and machines of one's own. It is a driver: it runs the preprocessor (**cpp**), **unproto** (which turns ANSI C prototypes into what the compiler reads), the compiler (**cc1**), the peephole optimizer (**copt**, with **-O**), the assembler **as09**(1) and the linker **ld09**(1), as each file needs, keeping the intermediate files in a temporary directory. By default it links a program with the start-up code and C library of the target that **-M** chooses.

The C is C89's, with a 16-bit **int** and pointers, a 32-bit **long**, IEEE single and double **float** and **double**, and an unsigned **char**; **_Bool** is C99's. The compiler doesn't know **const** and **volatile**: the C library's headers define them away (in features.h), so a program that uses them must include one of the headers (or **-D***const=*).

A file's suffix says what it is:

.c C source.

.i C already preprocessed.

.s Assembly, for **as09**(1).

.S, .x Assembly to be preprocessed first.

anything else An object file or library, for the linker.

OPTIONS

What to do

-c Compile or assemble each file to an object file (*name*.o), without linking.

-S Compile each C file to assembly (*name*.s).

-E Preprocess each file, to standard output (or the **-o** file).

-P Preprocess, without the line numbers.

-o *output* The output file: the program (a.out by default), or, with **-c**, **-S** or **-E**, the one file's.

-O Optimize: run each compiled file through **copt**, with its 6809 rules. **-O***name* also uses the rules in the file rules.*name*.

Targets

-M*target* The machine and system the program is for. Each defines a macro, and links its own start-up code and library:

- **s** Standalone (the default): returns to its loader (a monitor, or Color BASIC's EXEC); no I/O. Defines **___STANDALONE__**.
- **e** Color BASIC (or the Dragon's), loaded with LOADM or CLOADM and run with EXEC: I/O through the ROM's POLCAT and CHROUT, and, with Disk BASIC on a CoCo, its files. Defines **___CB___**.
- **r** A ROM cartridge for the CoCo (1 and 2) and the Dragon, which BASIC starts at $C000: the output is the ROM's image, laid out by the linker script *rom.ld* (the code and an image of the initialized data in the ROM, the data, BSS and heap in RAM from $0600, the stack below $8000), and I/O through the ROM's POLCAT and CHROUT, as with **e**; the start-up code copies the data to RAM, and **__exit** stops the program. Defines **___CB___** and **___CBROM___**.
- **o** OS-9 (and NitrOS-9): a position-independent program module, with its data reached through the U register (make the module with **objcopy09 -O os9**). Defines **___OS9___**.
- **f** FLEX: a command, a .CMD file (**objcopy09 -O flex**), with FLEX's terminal and files. Defines **___FLEX___** and **___FLEX9___**.
- **v** A Vectrex cartridge: the output is the ROM's image, laid out by the linker script *vectrex.ld* (the cartridge header at $0000, then the code and an image of the initialized data; the data, BSS and stack in the Vectrex's RAM, $C880 to $CBEA), with the BIOS's routines for drawing, the controllers and sound in *vectrex.h* and no other I/O; the start-up code copies the data to RAM, and **__exit** stops the program. The header (the title the BIOS shows) is libvec.a's, unless a module of the program's own defines **___vec__header** in segment 12. Defines **___VECTREX___**.
- **i** sim6809, the simulator bcc09's tests use. Defines **___SIM6809___**.
- **u, b** Color BASIC USR functions and Disk BASIC: they define **___CB___** but have no start-up code or library yet, so only **-x** links them, with the C library alone.

-x Link no start-up code: the program brings its own (the C library, and the target's library if it has one, are still linked).

Passing options on

-D*name* [=*value*]

-U*name*

-I*dir* Define or undefine a macro, or add an include directory, as **bcc09-cpp**(1) does.

-I (alone) Don't search the default include directory.

-L*dir* Add a library directory for the linker.

-L (alone) Don't search the default library directory.

-l*name* Link the library lib*name*.a. The linker's options **-s**, **-m**, **-i**, **-d**, **-y** and **-H***size* go to it too (see **ld09**(1)); its others, such as **-T***addr* , which **bcc09** uses for itself, are given with **-X**.

-X*option* Pass *option* to the linker, for example **-X-T0x3000** for the program's load address, or **-X-Sprog.ld** for a linker script (see **ld09**(1)).

-A*option* Pass *option* to the assembler.

-t *seg* Pass **-t** *seg* to the assembler.

-C*option* Pass *option* to the compiler (**cc1**).

-P*option* Pass *option* to the preprocessor.

-u*option* Pass *option* to **unproto**.

-W Show the assembler's warnings.

-kr Don't run **unproto** (for K&R C), and don't define **___STDC___**.

-ansi Accepted and ignored (unproto is the default).

-g, -p, -w Accepted and ignored.

Where things are

-B*prefix* Look for the passes in *prefix* first.

-M*/dir* The directory bcc09 is installed in, as **BCC_PREFIX** gives it.

-M- Find it from where the **bcc09** program is: its directory, less its *bin*.

-T*dir* The directory for temporary files (/tmp by default).

-v Print each command as it is run; twice, the temporary files' removal too; three times, the commands without running them. With no files, print the version (as in **bcc09 0.12.0**) and the usage message.

-V Print each file's name as it is compiled.

-e Use the compiler's own preprocessor in place of **cpp**.

MACROS

bcc09 defines **___BCC___**, **___6809___**, **___STDC___** (unless **-kr**), and the target's macro (see **-M**).

ENVIRONMENT

BCC_PREFIX The directory bcc09 is installed in: it runs its passes from *libexec/bcc09*, and finds the C library's headers in *share/bcc09/include* and its libraries in *share/bcc09* under it. By default, the directory bcc09 was built to be installed in (**make PREFIX=**...; /usr/local/bcc09 unless that says otherwise).

BCC_EXEC_PREFIX Another directory to look for the passes in.

FILES

***prefix*/libexec/bcc09** The passes: **cpp**, **unproto**, **cc1**, **copt** and its rules (rules.6809).

***prefix*/share/bcc09** The C library (libc.a), and each target's start-up code and library (crt0.o and libsa.a for **-Ms**; crt0.o and libcb.a for **-Me**; crtrom.o, librom.a and the linker script rom.ld for **-Mr**; crtvec.o, libvec.a and vectrex.ld for **-Mv**; crtos9.o and libos9.a, all of libc built for OS-9, for **-Mo**; crtflex.o and libflex.a for **-Mf**; crtsim.o and libsim.a for **-Mi**).

***prefix*/share/bcc09/include** The C library's headers.

EXIT STATUS

0 if every file was compiled (and the program linked), 1 otherwise.

EXAMPLES

A Color BASIC program, loaded at $3000, on a cassette:

```
bcc09 -Me -O -o hello -X-T0x3000 hello.c objcopy09 -O cas -n HELLO hello hello.cas
```

A ROM cartridge's image (for XRoar's **-cart-rom**, or an EPROM):

```
bcc09 -Mr -O -o hello.rom hello.c
```

A Vectrex cartridge, with a title of its own in *title.s* (see *vectrex.h*):

```
bcc09 -Mv -O -o game.bin game.c title.s
```

An OS-9 program module:

```
bcc09 -Mo -O -o hello.out hello.c objcopy09 -O os9 -n hello hello.out hello
```

NOTES

OS-9's, Color BASIC's and FLEX's lines end with a carriage return, and the C library passes bytes as they are: a program writes "\r" (on FLEX, "\r\n"), and reads such lines with **getdelim**(3) and '\r'. The C library's README describes each target in full.

SEE ALSO

as09(1), **ld09**(1), **ar09**(1), **objdump09**(1), **objcopy09**(1); the passes: **bcc09-cpp**(1), **bcc09-unproto**(1), **bcc09-cc1**(1), **bcc09-copt**(1)

as09(1)

NAME

as09 - assembler for the Motorola 6809

SYNOPSIS

as09 [**-guwk**] [**-o** *object*] [**-b** *binary*] [**-l** [*listing*]] [**-m** [*listing*]] [**-s** *symbols*] [**-n** *name*] [**-t** *seg*] [*source*]
as09 -v

DESCRIPTION

as09 assembles 6809 assembly language into an object file for **ld09**(1), or straight into a binary image. It is dev86's as86 for the 6809, and the assembler that **bcc09**(1) runs on the compiler's output. With no *source*, or **-**, it reads standard input.

A flag option is turned off by following it with **-**: **-w-** shows the warnings that **-w** hides.

OPTIONS

-o ***object*** Write an object file, for the linker.

-b ***binary*** Write a binary image: the assembled bytes, at the addresses **ORG** gives them, with no symbols to link. Without **-s**, the image starts at address 0 (zeros up to the first byte); with it, at the first byte, and the symbol file starts with the image's start and end (**$start** and **$end**).

-l [***listing***] Write a listing (to standard output if no file is named).

-m [***listing***] Write a listing with the lines that macros expand to.

-s ***symbols*** Write the symbol table to a file.

-n ***name*** The module's name in the object file (and in error messages), in place of the source file's.

-g Put only the global symbols (those exported or imported) in the object file.

-u Take an undefined symbol as imported, as if **IMPORT** named it (rather than an error).

-w Don't print warnings.

-k Keep the output files even if there are errors.

-t ***seg*** Make **.TEXT** select segment *seg*+3, rather than 0.

-v Print the version (as in **as09 0.12.0**) on standard output, and exit with status 0.

SOURCE FORMAT

A line has a label (starting in the first column, with or without a colon), an operation and its operands, separated by blanks or tabs, and a comment after a **;**. A line starting with ***** or **;** is a comment. Upper and lower case are the same in operations and register names, but not in symbols.

Numbers are decimal, $**hex** or **0x**hex, **%**binary, or octal with a leading 0; **'***c* is the character *c*'s code. Expressions have +, **-**, *****, /, **&**, |, «, », unary **-** and ~, and parentheses. < before an operand makes it direct (a page-0 address, through DP), > extended; an indexed operand is **,***reg* , *offset***,***reg* , **,X+**, **,-X** and so on, with **,PC** or **,PCR** for one relative to the program counter, and [...] for one indirect.

Pseudo-operations

.TEXT, .DATA, .BSS, LOC *n* Assemble into the code, the data, the BSS (which take no room in the object file, and are cleared at start-up), or segment *n* (0 to 15; code is 0, data and BSS 3; see **ld09**(1) for where the linker puts each, and its **___seg*n*DL** and **___seg*n*DH** symbols for each one's start and end).

FCB, .BYTE, DB *value,...* Bytes.

FDB, .WORD, DW *value,...* Words, high byte first.

FCC, .ASCII *"text"* The characters of a string (in single or double quotes).

.ASCIZ *"text"* The same, then a 0.

RMB, .BLKB, .SPACE *n* Leave *n* bytes; **.BLKW** *n* leaves *n* words.

symbol **EQU** *value*
symbol **SET** *value*

: Define a symbol (SET can define it again; EQU can't).

EXPORT, PUBLIC, GLOBAL *symbol,...* Make symbols this module defines visible to others.

IMPORT, EXTERN *symbol,...* Name symbols that other modules define.

ENTRY *symbol* Make *symbol* an entry: the linker loads the modules that the entry symbols need (only those), and the first entry it reads is the program's entry point.

symbol **COMM** *size*
.COMM *symbol*,*size*

: A common block, in the BSS: one, shared by every module that names it.

ORG *address* The address to assemble at (for **-b**).

SETDP *page* Tell the assembler what DP holds, for direct addressing.

IF, ELSE, ELSEIF, ENDIF Assemble lines or not, by an expression (IFC and ELSEIFC compare strings).

MACRO *name* Define a macro, up to **MEND**; *name* then assembles its lines.

INCLUDE, GET *file* Assemble another file here.

.ALIGN *n*, .EVEN Align the location.

FAIL, .WARN An error, or a warning.

END The end of the source.

EXIT STATUS

0 if there were no errors, 1 otherwise.

SEE ALSO

bcc09(1), **ld09**(1), **ar09**(1), **objdump09**(1)

ld09(1)

NAME

ld09 - linker for the Motorola 6809

SYNOPSIS

ld09 [**-MZdimstvy**[**-**]] [**-o** *output*] [**-T** *address*] [**-D** *address*] [**-H** *size*] [**-S** *script*] [**-A** *symbol=value*] [**-L** *dir*] [**-C** *name*] [**-l** *name*] [**-O** *file*] *file...*
ld09 -P [**-di**] [**-T** *address*] [**-D** *address*] [**-H** *size*]
ld09 -r -o *archive object...*

DESCRIPTION

ld09 links object files from **as09**(1), and members of libraries (archives made by **ar09**(1)), into an executable: an a.out file, which **objcopy09**(1) turns into what a machine loads. It is dev86's ld86 for the 6809; the compiler driver **bcc09**(1) runs it.

ld09 loads only the modules that something refers to, from the entry symbols (see **ENTRY** in **as09**(1)) on: an object file named on the command line, like a library's member, is left out if nothing needs it. A common block (an uninitialized variable from the compiler) that another module has too is no reason to load that module; a common block's storage comes with a loaded module that has one, as large as the largest of any module's. A library is searched only for what is undefined when it is read, so libraries come after the files that use them.

A linker script says where everything goes: which parts of which modules go into each output section, where the sections are, and symbols to define. Without **-S**, **ld09** uses a built-in default script, which gives the layout under SEGMENTS; **-P** prints it, as a start for one's own. See LINKER SCRIPTS.

OPTIONS

-o *output* The executable (a.out by default).

-T*address* The address the code is linked at (0 by default).

-D*address* The address the data is linked at, rather than straight after the code; it makes the data separate (**-i**).

-H*size* Room after the BSS, for the heap (or OS-9's stack: see **objcopy09**(1)).

The addresses and the size are hexadecimal, with or without a **0x** (so **-T3000** and **-T0x3000** are the same). With a script, **-T** and **-D** move its sections named **.text** and **.data**.

-i Separate code and data: the data is linked at 0 (or at **-D**'s address) instead of after the code, as OS-9's programs reach their data through a register.

-S *script* Link as the linker script in the file *script* says, rather than as the default script does.

-A *symbol=value* Define *symbol*, as if the script began with *symbol* = *value*; : the script can use it, and it takes the place of the script's **PROVIDE** of the same symbol. *value* is an expression, written as in a script.

-P Print the default script, for the **-T**, **-D**, **-H**, **-i** and **-d** given, and exit. Linking with it (**-S**) gives the same executable as linking without a script. No files are needed.

-Z Make input that the script doesn't place an error (see Orphans).

-l*name* Link the library lib*name*.a, from the library directories.

-L*dir* Search *dir* for libraries (and start-up files) too.

-C*name* Link the start-up file crt*name*.o, from the library directories, first.

-O*file* Link the library *file*, from the library directories.

-s Leave the symbols out of the executable.

-y Write each symbol's whole name, in the 8-character entries of an a.out symbol table and the entries after them (the default; **-y-** cuts names to 8 characters).

-d Write the code and data alone, with no a.out header (and no symbols).

-m Print the modules linked.

-M Print the layout (each output section, with its address, size and load address, and the input sections in it), then the symbols linked.

-t Trace the files and modules as they are read.

-r Make an archive of the objects, for **ar09**(1) or a later link, rather than linking them (with **-o** required).

-v Print the version (as in **ld09 0.12.0**) on standard output, and exit with status 0.

A flag option is turned off by following it with **-**, as in **-y-**.

SEGMENTS

This is the default script's layout. Of the 16 segments (**LOC** 0 to 15 in **as09**(1)), 0 and 1 are the code, placed at the text address. The data follows the code, or is at the data address (**-D**, or 0 with **-i**, after the code is padded to a multiple of 16 bytes): segment 2 (the direct page, moved to the start of the next 256-byte page if it would cross one), then 3 (the data, then the BSS) to 15. Each segment is followed by its common blocks (**COMM**, each rounded up to a multiple of 4 bytes), and each module's part of a segment is padded to a multiple of 4 bytes. The initialized data ends with the last segment that has any; the common blocks after it are the a.out's BSS, not in the file.

ld09 defines symbols for the program to find its parts: **___btext** and **___etext** (the code's start and end), **___bdata** and **___edata** (the data's), **___end** (the BSS's end), **___heap_top** (the end plus **-H**'s size), **___seg*n*DL** and **___seg*n*DH** (segment *n*'s start and end, *n* a hexadecimal digit), **___seg*n*CL** and **___seg*n*CH** (its common blocks'). **___btext**, **___etext**, **___bdata**, **___edata** and **___end** are always in the executable's symbol table (unless **-s**), where **objcopy09**(1) finds the load addresses.

LINKER SCRIPTS

A script is a list of commands. Comments are written `/* ... */` or start with #. For example, a program in ROM at $E000, with its initialized data stored in the ROM after the code but run in RAM at $0080 (where the start-up code copies it), and the vector table at $FFF0:

```
ENTRY(reset) OUTPUT_FORMAT(srec)

MEMORY
{
    ROM (rx) : ORIGIN = 0xE000, LENGTH = 0x1FF0
    VEC (r)  : ORIGIN = 0xFFF0, LENGTH = 16
    RAM (rw) : ORIGIN = 0x0080, LENGTH = 128
```

```
}

SECTIONS
{
    .text :
    {
        *(LOC0 LOC1)
    } > ROM = 0xFF

    .data :
    {
        __data_start = .;
        *(LOC3)
        __data_end = .;
    } > RAM AT> ROM

    .bss (NOLOAD) :
    {
        *(COMMON)
    } > RAM

    .vectors :
    {
        KEEP(vectors.o(LOC2))
    } > VEC

    __stack_top = ORIGIN(RAM) + LENGTH(RAM) - 1;
    __data_load = LOADADDR(.data);

    ASSERT(SIZEOF(.vectors) == 16, "vector table must be 16 bytes")
}
```

Commands

ENTRY(*symbol*) Start the program at *symbol*, before any **ENTRY** in the object files.

OUTPUT_FORMAT(*format*) **aout** (the default); **binary**, the code and data alone as with **-d**, covering the load addresses from the lowest to the highest used; **srec**, Motorola S-records; or **ihex**, Intel hex. S-records and Intel hex have one run of records for each section written, at its load address, without the gaps between sections, and the entry point in the end record; the S-records' header holds the output file's name.

MEMORY *{* name *[(*attributes*)]* : ORIGIN = *expr*, LENGTH = *expr ...* *}* Define memory regions, such as ROM and RAM. The attributes, such as **rx**, are only shown in the **-M** layout.

FILL(*byte*) The byte for gaps, where a section doesn't set its own (0 by default).

ASSERT(*expression*, "*message*") Stop with *message* if *expression* is 0 once the layout is complete. The modules that define its symbols are linked.

INCLUDE "*file*" Read another script here (also inside **SECTIONS**).

SECTIONS *{ ... }* Output sections and symbol assignments, in order.

***symbol* = *expression*;** Define a symbol.

PROVIDE(*symbol* = *expression*); Define it only if a linked module imports it and no module defines it.

HIDDEN(*symbol* = *expression*); Define a symbol that only the script can use.

Output sections

```
name [address] [(NOLOAD)] : [AT(lma)] [ALIGN(n)] [SUBALIGN(n)]
    { statements } [> region] [AT> region] [= fill]
```

Without an *address*, a section placed in a region (*> region*) starts at the region's next free address, and another starts at the current address (**.**), where the section before it ended. A section that doesn't fit in its region is an error.

A section is stored at its load address, which is its address unless **AT** gives another, or **AT>** allocates it from a region: initialized data that runs in RAM, for one, can be stored in ROM after the code, and copied by the start-up code. Separate load addresses need the **binary**, **srec** or **ihex** format.

A **NOLOAD** section takes up addresses but isn't written. It may only hold space reserved with **RMB** and common blocks: a module's initialized data, or **BYTE** or **WORD**, in one is an error, since it would be lost. **= fill** sets the byte for the gaps in the section. **ALIGN** rounds the start up to a multiple of *n*. **SUBALIGN** pads each input section in it to a multiple of *n* bytes.

The statements in a section, in order:

***file*(*sections*)** Add the matching input sections.

ONEPAGE *{ statements }* Keep the contents in one 256-byte page (as for a direct page): if they would end on a later page than they start, they move to the start of the next page.

symbol* = *expression*;, *PROVIDE*, *HIDDEN Define symbols, often with **.**, the current address.

. = *expression*; Move the current address forward, filling the gap with the fill byte.

KEEP(*file*(*sections*)) Add input sections as *file*(*sections*) does, and link their modules even if nothing refers to them.

BYTE(*expression*); WORD(*expression*); Put a byte, or a 16-bit word (high byte first), in the section.

FILL(*byte*); Use *byte* for the gaps that follow.

ASSERT(*expression*, "*message*"); As at the top level, but with **.** as here.

Input sections Each linked module has two input sections for each segment *n*: **LOC***n* , its code and data, and **COMMON***n* , its common blocks. **.text** means **LOC0**, **.data** means **LOC3** (which also holds as09's **.BSS**), and **COMMON** means all the common blocks.

file matches the object file's name, with or without its directory; a library's members are named *library*:*member* . Both *file* and the section names may have the wildcards ***** and **?**. Each input section goes to the first statement that matches it. A statement adds its input sections in the order of the modules, and each module's in the order of the segments.

A statement can start with **EXCLUDE_FILE(***file ...***)** , naming files (patterns, as above) that it doesn't take input from, also inside **KEEP**: **EXCLUDE_FILE(vectors.o) *(LOC0 LOC1)** takes segments 0 and 1 of every module but **vectors.o**, so that a later section can place it.

Orphans Input that no statement matches is added, with a warning, at the end of the last output section for its kind (segments 0 and 1, or the others) that takes input from all files (*****, without **EXCLUDE_FILE**), or else the last section for its kind. **-Z** makes it an error.

Expressions Expressions have C's operators, and numbers written as in C (with **0x** for hexadecimal, a leading 0 for octal), or with **$** for hexadecimal; a **K** after a number multiplies it by 1024. The functions are **ADDR(***section***)** , **SIZEOF(***section***)** , **LOADADDR(***section***)** , **ALIGN(***n***)** (the current address rounded up), **ALIGN(***expression***,** *n***)** , **DEFINED(***symbol***)** , **ORIGIN(***region***)** , **LENGTH(***region***)** , **MAX** and **MIN**. A symbol may be used before it is defined: the linker repeats the layout until nothing changes.

Symbols The symbols a script defines are absolute, and satisfy imports from any segment. A plain assignment to a symbol that a module also defines is an error.

Where a script doesn't assign them, **ld09** provides **___btext** and **___etext** (the a.out text's start and end), **___bdata** (its data's start), **___edata** (the end of the initialized data) and **___end** (the end of the data, BSS included), and writes them in the symbol table, for **objcopy09**(1). It also provides **___seg***n***DL** , **___seg***n***DH** , **___seg***n***CL** and **___seg***n***CH** if a module imports them: the start and end of where segment *n*'s input (or its common blocks) went, which must be in one place.

So a script for a C program needn't define anything for the C library. Its start-up code clears the memory from **___edata** to **___end** (so only uninitialized data, such as the common blocks, goes there), and calls the init functions, the addresses in segment 5, from **___seg5DL** to **___seg5DH**: segment 5 needs an input statement of its own, as ***(LOC5)** (a statement adds its input module by module, so in ***(LOC3 LOC5)** each module's segment 5 would follow its segment 3).

For each output section, the linker also provides **___***name***__start** , **___***name***__end** and **___***name***__load** (its load address), with the name's leading dot dropped, if a module imports them: start-up code can copy **.data** from **___data__load** to **___data__start**, for one.

a.out output The output sections up to the first one that can take input from segments 2 to 15 are the a.out's text, and the rest its data. The executable has separate code and data if the data doesn't start where the text ends. Uninitialized data at the end of the data, such as a **NOLOAD** section, is the a.out's BSS: it isn't in the file. The a.out header's total memory reaches **___heap__top** if the script sets it.

DIAGNOSTICS

A symbol defined in two linked modules is reported with a warning, as in "dup redefined in file libx.a(x.o); using definition in prog.o": the first definition read is used. A library member that isn't linked doesn't count, so a program can have its own version of a library function.

Input that a script doesn't place is reported with a warning, as in "segs.o(LOC5) isn't placed by the script; adding it to .data". Initialized data in a module's segment that a script puts in a **NOLOAD** section is an error, reported once for each module and segment, as in "noload.o: LOC4 has initialized data, but .bss is NOLOAD". A script's errors name its file and line.

EXIT STATUS

0 if the link succeeded; 1 after an error such as an undefined symbol; 2 after an error that stops the link at once, such as a file that can't be read, or a script's error.

EXAMPLES

A program for a monitor, loaded at $4000:

```
ld09 -T4000 -o prog.out -L/usr/share/bcc09 -C0 prog.o -lc -lsa
```

The default script, as a start for one's own, and a link with it (through **bcc09**(1), which passes the script with **-X**):

```
ld09 -P -T3000 > prog.ld bcc09 -Me -o prog -X-Sprog.ld prog.c
```

SEE ALSO

bcc09(1), **as09**(1), **ar09**(1), **objdump09**(1), **objcopy09**(1)

The bcc09 User's Manual (*manual.pdf*).

HISTORY

Derived from **ld86** in Bruce Evans' C compiler, as the dev86 package has it. The linker scripts come from **ld01** in the mc6801-tools project.

ar09(1)

NAME

ar09 - make and change libraries (archives) of object files

SYNOPSIS

ar09 [**-**] *key* [*position*] *archive* [*file...*]

DESCRIPTION

ar09 keeps object files from **as09**(1) together in an archive, a library that **ld09**(1) links the members of that a program needs. It is the Free Software Foundation's ar of 1988, as dev86 has it.

The *key* is one operation letter, with modifier letters after it.

OPERATIONS

r Put the files in the archive, replacing members of the same names (made if there is no archive).

q Append the files to the archive quickly, without looking for members of the same names.

d Delete the members named.

x Extract the members named (all of them if none is).

t List the members (those named, or all).

p Print the members' contents on standard output.

m Move the members named, in the order named, to the end of the archive (or to *position*, with **a**, **b** or **i**).

MODIFIERS

a Put new or moved members after the member *position*.

b, i Put them before *position*.

c Make the archive without saying so.

u With **r**, replace only members older than their files.

o With **x**, give the files the dates the members have.

s Meant to write a symbol table into the archive, as the member ___.SYMDEF: it can't read **as09**'s objects, so the table is empty. **ld09** doesn't need one (it reads every member's symbols), and ignores it.

v Say what is done to each member.

l Accepted and ignored.

EXIT STATUS

0 if all went well, 1 otherwise.

EXAMPLES

A library of three objects, then its members:

ar09 rc libmine.a one.o two.o three.o ar09 t libmine.a

SEE ALSO

bcc09(1), **as09**(1), **ld09**(1), **objdump09**(1)

objdump09(1)

NAME

objdump09, nm09, size09 - show what is in 6809 object files, libraries and executables

SYNOPSIS

objdump09 [**-n**|**-s**] [**-o**] *file...*
nm09 [**-o**] *file...*
size09 *file...*

DESCRIPTION

objdump09 reads object files from **as09**(1), libraries from **ar09**(1) (each member in turn), and executables from **ld09**(1). It is dev86's objdump86; called **nm09** it lists symbols, and called **size09** sizes, as **-n** and **-s** make it do.

By default it shows everything: an object file's module name, its symbols, its segments' sizes and its code, with the relocations; an executable's header (entry point, flags), sizes and symbols.

OPTIONS

-n List the symbols, as **nm**(1) does: each one's value, a letter for its kind, and its name. The letter is **T** or **t** for code (segments 0 and 1), **D** or **d** for data, **a** for an absolute value, **b** for a common block that is local, **C** for a common block, and **U** for a symbol that is undefined (imported); upper case is exported.

-s Show the sizes of the code (text), data and BSS, as **size**(1) does, with a total line for several files.

-o With **-n**, put the file's name before each symbol.

EXIT STATUS

0, or 1 if no file is named.

SEE ALSO

bcc09(1), **as09**(1), **ld09**(1), **ar09**(1), **objcopy09**(1)

objcopy09(1)

NAME

objcopy09 - make a 6809 executable into what a machine loads

SYNOPSIS

objcopy09 [**-O** *format*] [**-n** *name*] [**-T** *address*] [**-D** *address*] [**-p**] [**-v** *revision*] [**-d** *size*] [**-r**] *executable output*

DESCRIPTION

objcopy09 reads an executable (an a.out file) from **ld09**(1) and writes its code and data in one of the forms that 6809 machines, their systems and their tools load.

OPTIONS

-O *format* The form to write:

binary The bytes alone (the default): the code, then the data.

srec Motorola S-records: S0 (holding the name), S1, and S9 with the entry point.

ihex Intel hex records, with CR LF line ends.

decb A Disk Extended Color BASIC file, for LOADM (and EXEC): blocks of an address and bytes, a second for data linked apart from the code, and the entry point.

ddos A DragonDOS binary file (code and data together).

cas A Color BASIC cassette image, for CLOADM: a name block (the name, up to 8 characters, and a machine-language file's entry point and load address), data blocks, and an end block, each after a leader. For the CoCo and the Dragon, and for emulators (XRoar's **-load-tape**).

wav The same, as the sound a cassette recorder plays to the machine (43200 samples a second).

os9 An OS-9 program module: the module header (type $11, a 6809 program), the code, the image of the data (which the start-up code of **bcc09 -Mo** copies to the data area), the name, and the CRC. The data area the module asks for holds the program's data and BSS and then **-d** bytes for the stack. The executable's data should be linked at 0 (**ld09 -i**, as **bcc09 -Mo** does): there is a warning if it isn't.

flex A FLEX binary (.CMD) file, a FLEX command: $02 records of an address, a count (up to 255) and bytes, and a $16 record with the entry point.

-n *name* The name: of the cassette file, the OS-9 module, or the S0 record. By default the output file's name, in upper case.

-T *address* The code's load address (by default the address it was linked at).

-D *address* The data's load address, for data linked apart from the code (by default the address it was linked at).

-p With **binary**, pad the gap between the code and the data with zeros, so that the data is at its address's place in the file.

-v *revision* With **os9**, the module's revision, the low 4 bits of its header's attribute and revision byte (0 to 15; up to 127 is taken, setting attribute bits too).

-d *size* With **os9**, the bytes of the data area for the stack (by default the size of **ld09 -H**, or 1024).

-r With **os9**, mark the module reentrant (it is anyway if its data is at 0, or it has none).

Addresses and sizes are decimal, hexadecimal after **0x** or **$**, or octal after a 0, and must fit in 16 bits.

EXIT STATUS

0 if the file was written, 1 otherwise.

EXAMPLES

A Color BASIC program on a cassette, a FLEX command and an OS-9 module:

objcopy09 -O cas -n HELLO hello hello.cas objcopy09 -O flex hello.out HELLO.CMD objcopy09 -O os9 -n hello -d 2048 hello.out hello

SEE ALSO

bcc09(1), **ld09**(1), **objdump09**(1)

bcc09-cpp(1)

NAME

bcc09-cpp - the C preprocessor of bcc09 (cpp)

SYNOPSIS

prefix **/libexec/bcc09/cpp** [**-A**|**-K**] [**-P**] [**-D** *name*[=*value*]] [**-U** *name*] [**-I** *dir*] [**-o** *output*] *file*

DESCRIPTION

cpp is the C preprocessor that **bcc09**(1) runs first on each C file (and on .S and .x assembly files): it includes files, defines and expands macros, and keeps or drops lines by **#if**, **#ifdef** and the rest, as C89 says, writing the result to *output* (or standard output) with # lines that give each line's file and number, as **#line** can too. It is dev86's cpp; one normally runs it through **bcc09 -E**, which gives it the include directories and macros for its target.

A directive that it can't make sense of is an error, naming the file, the line and the directive.

OPTIONS

-A ANSI C (the driver gives it unless **-kr**).

-K K&R C: no **___DATE___** and **___TIME___**.

-P Leave out the # lines that give the line numbers.

-D*name* [=*value*] Define a macro (as 1 if there is no value).

-U*name* Undefine a macro.

-I*dir* Look for #include files in *dir* too.

-o *output* The output file.

-O Define **___OPTIMISED___**.

-c Define **___CALLER_SAVES___**.

-f Define **___FIRST_ARG_IN_AX___**.

-0, -3 Define the macros of dev86's 8086 and 80386 compilers (no use for the 6809).

-d Print the tokens, for debugging.

-T Pass on only some of the tokens (dev86's; not used).

-C Not done (comments are always removed).

EXIT STATUS

0, or 1 if there were errors.

SEE ALSO

bcc09(1), **bcc09-unproto**(1), **bcc09-cc1**(1)

bcc09-unproto(1)

NAME

bcc09-unproto - compile ANSI C with traditional UNIX C compiler (bcc09's unproto)

PACKAGE

`unproto`

SYNOPSIS

```
/somewhere/cpp ...

cc cflags -E file.c | unproto >file.i; cc cflags -c file.i
```

DESCRIPTION

This document describes a filter that sits in between the UNIX C preprocessor and the next UNIX C compiler stage, on the fly rewriting ANSI-style syntax to old-style syntax. Typically, the program is invoked by the native UNIX C compiler as an alternate preprocessor. The unprototyper in turn invokes the native C preprocessor and massages its output. Similar tricks can be used with the lint(1) command.

Language constructs that are always rewritten:

function headings, prototypes, pointer types ANSI-C style function headings, function prototypes, function pointer types and type casts are rewritten to old style. <stdarg.h> support is provided for functions with variable-length argument lists.

character and string constants The \a and \x escape sequences are rewritten to their (three-digit) octal equivalents.

Multiple string tokens are concatenated; an arbitrary number of whitespace or comment tokens may appear between successive string tokens.

Within string constants, octal escape sequences are rewritten to the three-digit \ddd form, so that string concatenation produces correct results.

date and time The ___DATE___ and ___TIME___ tokens are replaced by string constants of the form "Mmm dd yyyy" and "hh:mm:ss", respectively. The result is subjected to string concatenation, just like any other string constant.

Language constructs that are rewritten only if the program has been configured to do so:

void types The unprototyper can be configured to rewrite "void *" to "char *", and even to rewrite plain "void" to "int". These features are configurable because many traditional UNIX C compilers do not need them.

Note: (void) argument lists are always replaced by empty ones.

ANSI C constructs that are not rewritten because the traditional UNIX C preprocessor provides suitable workarounds:

const and volatile Use the "-Dconst=" and/or "-Dvolatile=" preprocessor directives to get rid of unimplemented keywords.

token pasting and stringizing The traditional UNIX C preprocessor provides excellent alternatives. For example:

```
#define string(bar)     "bar"           /* instead of: # x */
#define paste(x,y)      x/**y         /* instead of: x##y */
```

There is a good reason why the # and ## operators are not implemented in the unprototyper. After program text has gone through a non-ANSI C preprocessor, all information about the grouping of the operands of # and ## is lost. Thus, if the unprototyper were to perform these operations, it would produce correct results only in the most trivial cases. Operands with embedded blanks, operands that expand to null tokens, and nested use of # and/or ## would cause all kinds of obscure problems.

Unsupported ANSI features:

trigraphs and #pragmas Trigraphs are useful only for systems with broken character sets. If the local compiler chokes on #pragma, insert a blank before the "#" character, and enclose the offending directive between #ifdef and #endif.

SEE ALSO

cc(1), how to specify a non-default C preprocessor. Some versions of the lint(1) command are implemented as a shell script. It should require only minor modification for integration with the unprototyper. Other versions of the lint(1) command accept the same command syntax as the C compiler for the specification of a non-default preprocessor. Some research may be needed.

FILES

`/wherever/stdarg.h, provided with the unproto filter.`

DIAGNOSTICS

Problems are reported on the standard error stream. A non-zero exit status means that there was a problem.

BUGS

The unprototyper should be run on preprocessed source only: unexpanded macros may confuse the program.

Declarations of (object) are misunderstood and will result in syntax errors: the objects between parentheses disappear.

Sometimes does not preserve whitespace after parentheses and commas. This is a purely aesthetical matter, and the compiler should not care. Whitespace within string constants is, of course, left intact.

Does not generate explicit type casts for function-argument expressions. The lack of explicit conversions between integral and/or pointer argument types should not be a problem in environments where sizeof(int) == sizeof(long) == sizeof(pointer). A more serious problem is the lack of automatic type conversions between integral and floating-point argument types. Let lint(1) be your friend.

BCC09

In bcc09, **unproto** runs between **bcc09-cpp**(1) and **bcc09-cc1**(1) (the driver runs it, as **unproto** *input output* , unless **-kr**; it has no options), built with two of its own options (see its Makefile):

__PROTO__ After every () that it writes for a prototype (in declarations, function pointer types, casts, typedefs and parameters), and after the argument names of a function heading that has a prototype's form, it writes **___proto(***type name, ...***)** , from which **cc1** knows the parameters' types: a call converts each argument to its parameter's type, as C89 says, and a call with the wrong number of arguments is an error.

__VA__ALIST__ In the heading of a function with a variable argument list, an **int** parameter named **___va__alist** takes the place of the ..., for **stdarg.h**'s **va__start** to take the address of (the first argument is passed in a register, so the last named one can't lead to the others).

The BUGS above are dev86's: with **__PROTO__** arguments are converted, between floating point types too.

AUTHOR(S)

Wietse Venema (wietse@wzv.win.tue.nl)
Eindhoven University of Technology
Department of Mathematics and Computer Science
Den Dolech 2, P.O. Box 513, 5600 MB Eindhoven, The Netherlands

LAST MODIFICATION

93/06/18 22:29:37

VERSION/RELEASE

1.6

bcc09-cc1(1)

NAME

bcc09-cc1 - the compiler proper of bcc09 (cc1)

SYNOPSIS

prefix **/libexec/bcc09/cc1** [**-ptuwdOP**[**-**]] [**-E** [**-D** *name*] [**-U** *name*] [**-I** *dir*]] [**-o** *output*] [*file*]

DESCRIPTION

cc1 compiles preprocessed C (as **bcc09-cpp**(1) and **bcc09-unproto**(1) leave it) to 6809 assembly for **as09**(1), from *file* (or standard input) to *output* (or standard output). It is dev86's bcc compiler, for the 6809; **bcc09**(1) runs it, and its **-C** option passes options to it.

A function's first argument is passed in X (a long's in D and X), and the others on the stack, above the return address, which the caller removes; a float is passed as a double, and a double always on the stack. An int or pointer result is in X, a long's or a float's in D and X (the high word in D), and a double's in D, X, Y and U. The caller saves the registers it needs: a function may change them all (but U under **-u**, the data's). The C library's README describes the conventions in full.

Its own preprocessor (**-E**) is what **bcc09 -e** uses in place of **bcc09-cpp**(1).

OPTIONS

A flag option is turned off by following it with **-**.

-p Position-independent code: functions and string constants reached relative to the program counter, calls with **LBSR**.

-u That, and the data reached through U, linked at 0 (OS-9's way; **bcc09 -Mo** gives it): U is then not used for values, and a static variable that holds an address is set by code that runs before main.

-t Put each line of the C source in the assembly, as a comment.

-d Put the compiler's expression trees in the assembly, as comments.

-w Print the location counter after each line (and don't make branches short).

-O Define **___OPTIMISED___**; it doesn't change the code for the 6809 (the driver's **-O** runs **bcc09-copt**(1)).

-c, -f Callers save registers, and the first argument goes in a register: always so for the 6809.

-E Preprocess only.

-D*name*
-U*name*
-I*dir*

: With **-E**, as **bcc09-cpp**(1) takes them.

-P With **-E**, no line numbers.

-o *output* The output file.

MACROS

With **-E**, cc1 defines **___BCC___** and **___AS09___**, and **___CALLER_SAVES___** with **-c**, **___FIRST_ARG_IN_X___** with **-f** and **___POS_INDEPENDENT___** with **-p**; **___FILE___** and **___LINE___** are built in.

EXIT STATUS

0, or 1 if there were errors (given in the output, as a **fail** line and a comment, and on standard error).

SEE ALSO

bcc09(1), **bcc09-cpp**(1), **bcc09-unproto**(1), **bcc09-copt**(1), **as09**(1)

bcc09-copt(1)

NAME

bcc09-copt - the peephole optimizer of bcc09 (copt)

SYNOPSIS

prefix **/libexec/bcc09/copt** [**-c** *char*] [**-f** *input*] [**-o** *output*] [**-b** *n*] [**-h** *text*] [**-d** *dir*] *rules...*

DESCRIPTION

copt reads assembly (the output of **bcc09-cc1**(1)) and replaces sequences of lines that a rule describes with that rule's replacement, file of rules after file of rules, writing the result. **bcc09 -O** runs it with the 6809's rules, rules.6809, which make code smaller. It is Gero Kuhlmann's copt, from dev86.

OPTIONS

-f*input* Read *input* (standard input by default).

-o *output* Write *output* (standard output by default).

-c*char* Lines starting with *char* are comments: they don't break a sequence of lines a rule matches (and are dropped from a match). **bcc09** gives **-c;**.

-d*dir* Read the rules files from *dir*.

-b*n* After a replacement, go back *n* lines before trying the rules again.

-h*text* Write *text* as the output's first line.

RULES

A rules file holds rules, each the lines to match, a line =, and the lines to replace them with, up to an empty line; a line starting with # is a comment. The rules are tried in order at each line of the input. A rule's lines must match consecutive lines (a label, a line not indented, ends a sequence); in them:

%*n* (a digit) matches any text, which the same %*n* must match again in the rule, and which %*n* in the replacement stands for; **%*** matches any text, to be forgotten.

%[*a*|*b*|*...*]*n* matches one of the texts, as %*n* .

%![*a*|*b*|*...*] matches if none of the texts is there.

%(*expression*) matches a number equal to the expression, in which %*n* is a variable's value; in a replacement it is the expression's value, in hexadecimal. A rule whose expression isn't a number (an address, such as a symbol's offset) doesn't apply.

%=[*a*|*b*][*c*|*d*]*n* in a replacement, the text of the second list at the place where the variable's text is in the first.

%% is %.

A replacement can't be empty: an empty line there ends the rules.

EXAMPLES

A store and a load of the same place leave the value in the register, so the load goes (unless the address increments or decrements a register):

```
ST%1 %2,%[X|Y|U|S]3 LD%1 %2,%[X|Y|U|S]3 = ST%1 %2,%3
```

NOTES

rules.6809's rules take every address for memory: a load straight after a store to the same address goes. A program that writes a hardware register and reads it straight back (cc1 has no **volatile**) gets the value it wrote, not the register's: compile that code without **-O**.

EXIT STATUS

0, or 1 if a file can't be read or written.

SEE ALSO

bcc09(1), **bcc09-cc1**(1)